

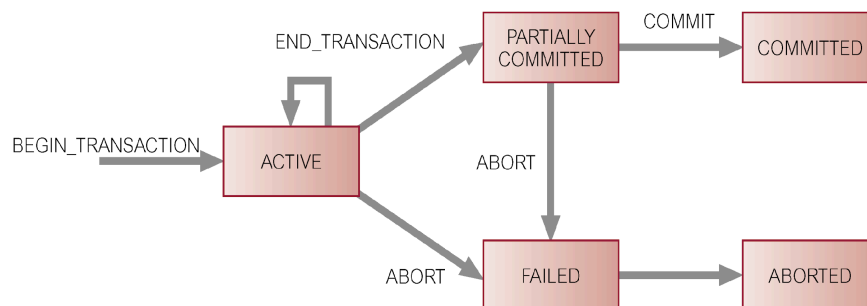
Transaction Management

Transaction

Action, or series of actions, carried out by user or application, which reads or updates contents of database.

- Logical unit of work on the database.
- Application program is series of transactions with non-database processing in between.
- Transforms database from one consistent state to another, although consistency may be violated during transaction.
- Can have one of two outcomes:
 - Success - transaction *commits* and database reaches a new consistent state.
 - Failure - transaction *aborts*, and database must be restored to consistent state before it started.
 - Such a transaction is *rolled back* or *undone*.
- Committed transaction cannot be aborted.
- Aborted transaction that is rolled back can be restarted later.

State Transition Diagram for Transaction



Properties of Transactions

Four basic (*ACID*) properties of a transaction are:

- **Atomicity:** 'All or nothing' property.
- **Consistency:** Must transform database from one consistent state to another.
- **Isolation:** Partial effects of incomplete transactions should not be visible to other transactions.
- **Durability:** Effects of a committed transaction are permanent and must not be lost because of later failure.

Concurrency Control

Process of managing simultaneous operations on the database without having them interfere with one another.

- Prevents interference when two or more users are accessing database simultaneously and at least one is updating data.
- Although two transactions may be correct in themselves, interleaving of operations may produce an incorrect result.

Need for Concurrency Control

Three examples of potential problems caused by concurrency:

- Lost update problem.
- Uncommitted dependency problem.
- Inconsistent analysis problem.

Lost Update Problem

- Successfully completed update is overridden by another user.
- T_1 withdrawing £10 from an account with bal_x, initially £100.
- T_2 depositing £100 into same account.
- Serially, final balance would be £190.

Time	T ₁	T ₂	bal _x
t ₁		begin_transaction	100
t ₂	begin_transaction	read(bal _x)	100
t ₃	read(bal _x)	bal _x = bal _x + 100	100
t ₄	bal _x = bal _x - 10	write(bal _x)	200
t ₅	write(bal _x)	commit	90
t ₆	commit		90

- Loss of T₂'s update avoided by preventing T₁ from reading bal_x until after update.

Uncommitted Dependency Problem

- Occurs when one transaction can see intermediate results of another transaction before it has committed.
- T₄ updates bal_x to £200 but it aborts, so bal_x should be back at original value of £100.
- T₃ has read new value of bal_x (£200) and uses value as basis of £10 reduction, giving a new balance of £190, instead of £90.

Time	T ₃	T ₄	bal _x
t ₁		begin_transaction	100
t ₂		read(bal _x)	100
t ₃		bal _x = bal _x + 100	100
t ₄	begin_transaction	write(bal _x)	200
t ₅	read(bal _x)	;	200
t ₆	bal _x = bal _x - 10	rollback	100
t ₇	write(bal _x)		190
t ₈	commit		190

- Problem avoided by preventing T₃ from reading bal_x until after T₄ commits or aborts.

Inconsistent Analysis Problem

- Occurs when transaction reads several values but second transaction updates some of them during execution of first.
- Sometimes referred to as *dirty read* or *unrepeatable read*.
- T₆ is totaling balances of account x (£100), account y (£50), and account z (£25).
- Meantime, T₅ has transferred £10 from bal_x to bal_z, so T₆ now has wrong result (£10 too high).

Time	T ₅	T ₆	bal _x	bal _y	bal _z	sum
t ₁		begin_transaction	100	50	25	
t ₂	begin_transaction	sum = 0	100	50	25	0
t ₃	read(bal _x)	read(bal _x)	100	50	25	0
t ₄	bal _x = bal _x - 10	sum = sum + bal _x	100	50	25	100
t ₅	write(bal _x)	read(bal _y)	90	50	25	100
t ₆	read(bal _z)	sum = sum + bal _y	90	50	25	150
t ₇	bal _z = bal _z + 10		90	50	25	150
t ₈	write(bal _z)		90	50	35	150
t ₉	commit	read(bal _z)	90	50	35	150
t ₁₀		sum = sum + bal _z	90	50	35	185
t ₁₁		commit	90	50	35	185

- Problem avoided by preventing T₆ from reading bal_x and bal_z until after T₅ completed updates.

Serializability

- Objective of a concurrency control protocol is to schedule transactions in such a way as to avoid any interference.

- Could run transactions serially, but this limits degree of concurrency or parallelism in system.
- Serializability identifies those executions of transactions guaranteed to ensure consistency.

Schedule

Sequence of reads/writes by set of concurrent transactions.

Serial Schedule

Schedule where operations of each transaction are executed consecutively without any interleaved operations from other transactions.

Nonserial Schedule

- Schedule where operations from set of concurrent transactions are interleaved.
- Objective of serializability is to find nonserial schedules that allow transactions to execute concurrently without interfering with one another.
- In other words, want to find nonserial schedules that are equivalent to *some* serial schedule. Such a schedule is called *serializable*.

In serializability, ordering of read/writes is important:

- (a) If two transactions only read a data item, they do not conflict and order is not important.
- (b) If two transactions either read or write completely separate data items, they do not conflict and order is not important.
- (c) If one transaction writes a data item and another reads or writes same data item, order of execution is important.

Concurrency Control Techniques

- Two basic concurrency control techniques:
 - Locking,
 - Timestamping.
- Both are conservative approaches: delay transactions in case they conflict with other transactions.
- Optimistic methods assume conflict is rare and only check for conflicts at commit.

Locking

Transaction uses locks to deny access to other transactions and so prevent incorrect updates.

- Most widely used approach to ensure serializability.
- Generally, a transaction must claim a *shared (read)* or *exclusive (write)* lock on a data item before read or write.
- Lock prevents another transaction from modifying item or even reading it, in the case of a write lock.

Locking - Basic Rules

- If transaction has shared lock on item, can read but not update item.
- If transaction has exclusive lock on item, can both read and update item.
- Reads cannot conflict, so more than one transaction can hold shared locks simultaneously on same item.
- Exclusive lock gives transaction exclusive access to that item.
- Some systems allow transaction to upgrade read lock to an exclusive lock, or downgrade exclusive lock to a shared lock.

Two-Phase Locking (2PL)

Transaction follows 2PL protocol if all locking operations precede first unlock operation in the transaction.

Two phases for transaction:

- Growing phase - acquires all locks but cannot release any locks.
- Shrinking phase - releases locks but cannot acquire any new locks.

There is no requirement that all locks be obtained simultaneously. Normally, the transaction acquires some locks, does some processing, and goes on to acquire additional locks as needed. However, it never releases any lock until it has reached a stage where no new locks are needed. The rules are:

A transaction must acquire a lock on an item before operating on the item. The lock may be read or write, depending on the type of access needed.

Once the transaction releases a lock, it can never acquire any new locks.

If upgrading of locks is allowed, upgrading can take place only during the growing phase and may require that the transaction wait until another transaction releases a shared lock on the item. Downgrading can take place only during the shrinking phase. We now look at how two-phase locking is used to resolve the three problems identified before:

Preventing Lost Update Problem using 2PL

Time	T ₁	T ₂	bal _x
t ₁		begin_transaction	100
t ₂	begin_transaction	write_lock(bal _x)	100
t ₃	write_lock(bal _x)	read(bal _x)	100
t ₄	WAIT	bal _x = bal _x + 100	100
t ₅	WAIT	write(bal _x)	200
t ₆	WAIT	commit/unlock(bal _x)	200
t ₇	read(bal _x)		200
t ₈	bal _x = bal _x - 10		200
t ₉	write(bal _x)		190
t ₁₀	commit/unlock(bal _x)		190

Preventing Uncommitted Dependency Problem using 2PL

Time	T ₃	T ₄	bal _x
t ₁		begin_transaction	100
t ₂		write_lock(bal _x)	100
t ₃		read(bal _x)	100
t ₄	begin_transaction	bal _x = bal _x + 100	100
t ₅	write_lock(bal _x)	write(bal _x)	200
t ₆	WAIT	rollback/unlock(bal _x)	100
t ₇	read(bal _x)		100
t ₈	bal _x = bal _x - 10		100
t ₉	write(bal _x)		90
t ₁₀	commit/unlock(bal _x)		90

Preventing Inconsistent Analysis Problem using 2PL

Time	T ₅	T ₆	bal _x	bal _y	bal _z	sum
t ₁		begin_transaction	100	50	25	
t ₂	begin_transaction	sum = 0	100	50	25	0
t ₃	write_lock(bal _x)		100	50	25	0
t ₄	read(bal _x)	read_lock(bal _x)	100	50	25	0
t ₅	bal _x = bal _x - 10	WAIT	100	50	25	0
t ₆	write(bal _x)	WAIT	90	50	25	0
t ₇	write_lock(bal _z)	WAIT	90	50	25	0
t ₈	read(bal _z)	WAIT	90	50	25	0
t ₉	bal _z = bal _z + 10	WAIT	90	50	25	0
t ₁₀	write(bal _z)	WAIT	90	50	35	0
t ₁₁	commit/unlock(bal _x , bal _z)	WAIT	90	50	35	0
t ₁₂		read(bal _x)	90	50	35	0
t ₁₃		sum = sum + bal _x	90	50	35	90
t ₁₄		read_lock(bal _y)	90	50	35	90
t ₁₅		read(bal _y)	90	50	35	90
t ₁₆		sum = sum + bal _y	90	50	35	140
t ₁₇		read_lock(bal _z)	90	50	35	140
t ₁₈		read(bal _z)	90	50	35	140
t ₁₉		sum = sum + bal _z	90	50	35	175
t ₂₀		commit/unlock(bal _x , bal _y , bal _z)	90	50	35	175

Timestamping

- Transactions ordered globally so that older transactions, transactions with *smaller* timestamps, get priority in the event of conflict.
- Conflict is resolved by rolling back and restarting transaction.
- No locks so no deadlock.

Timestamp

A unique identifier created by DBMS that indicates relative starting time of a transaction.

- Can be generated by using system clock at time transaction started, or by incrementing a logical counter every time a new transaction starts.
- Read/write proceeds only if *last update on that data item* was carried out by an older transaction.
- Otherwise, transaction requesting read/write is restarted and given a new timestamp.
- Also timestamps for data items:
 - read-timestamp - timestamp of last transaction to read item;
 - write-timestamp - timestamp of last transaction to write item.

With timestamping, if a transaction attempts to read or write a data item, then the read or write is only allowed to proceed if the last update on that data item was carried out by an older transaction. Otherwise, the transaction requesting the read/write is restarted and given a new timestamp. New timestamps must be assigned to restarted transactions to prevent their being continually aborted and restarted. Without new timestamps, a transaction with an old timestamp might not be able to commit owing to younger transactions having already committed.